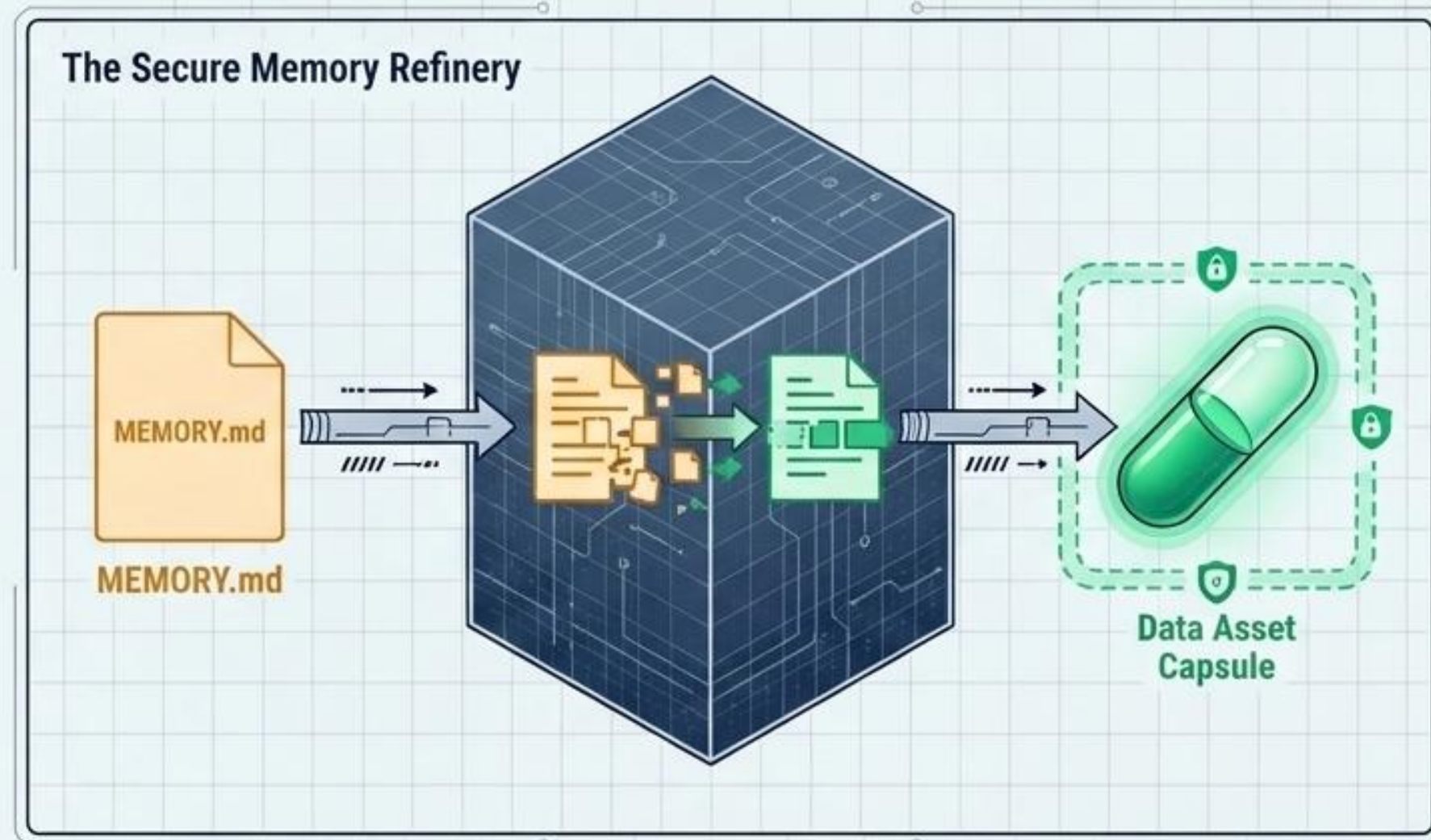


手机智能体个性化 记忆管理的隐私保护 与合规共享研究

从可检索上下文，
到可治理数据资产

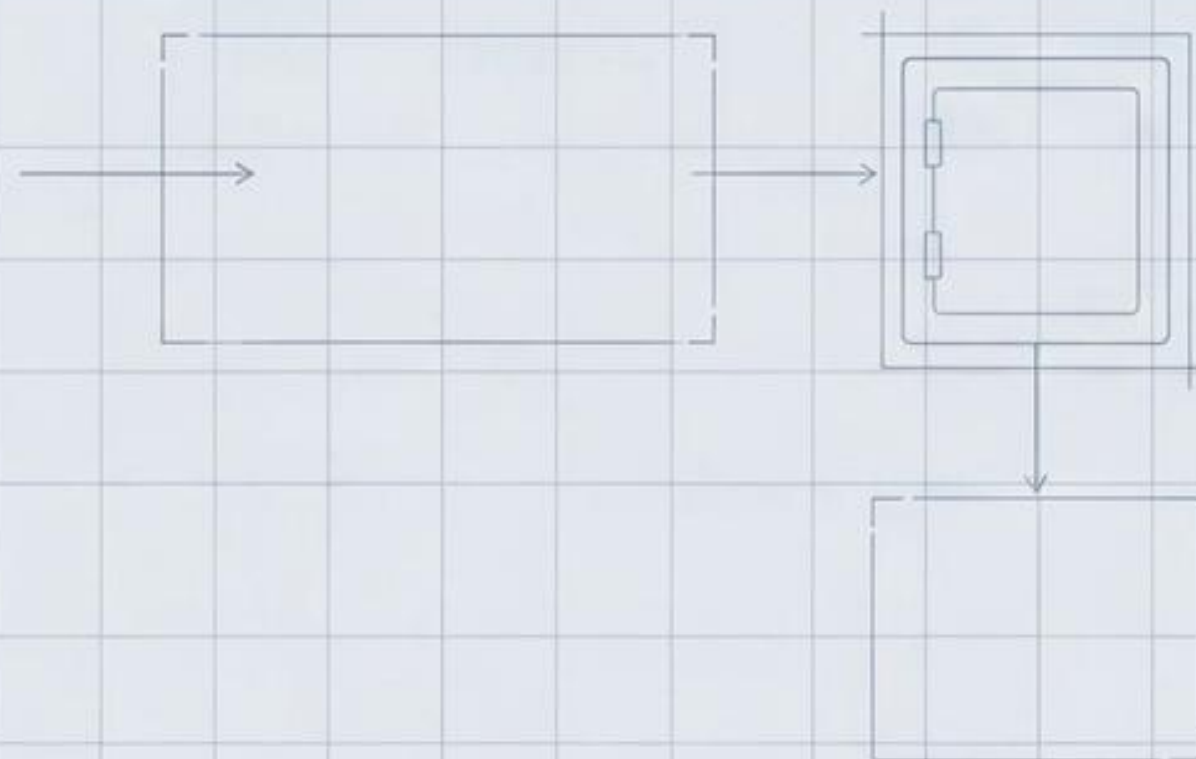
怎么在不失控的前提下，让智能体记得更久、用得更准、流动得更安全？



分享人：数智化部研创中心 李华福 | 2026年6月

本次分享目录

1. 背景与问题
2. 总体架构
3. 技术方案与创新点
4. 原型验证
5. 总结与展望



背景与问题：长期记忆正在从能力问题变成治理问题

随着手机智能体的发展，其核心竞争力正从“即时对话能力”转向“长期个性化服务能力”。这一转变的基础，正是构建一个可持续、可检索、可治理的记忆系统。



即时对话能力

一次任务内使用上下文，生命周期短，仅服务单次交互。



长期个性化能力

沉淀用户偏好与任务习惯，反复影响未来任务，提供千人千面的体验。



持久化记忆资产

记忆内容落到文件、索引或向量库中，成为可多次复用的数据资产。



跨域流动风险

跨设备、跨应用、第三方协作场景会放大隐私边界与数据管理问题。



核心转变

记忆不再是一次性的上下文（Context），而是持久化的数据（Persistent Data）。当记忆可以被长期长期保存、反复索引、跨”场景召回和跨设备同步时，它就从一个“能力问题”转变为一个“治理问题”。

OpenClaw 默认记忆机制： 单一可信操作者边界

OpenClaw将记忆外显化为明文工作区文件，
并通过内建的SQLite与向量混合检索机制实现持续记忆



传统范式：记忆作为“上下文”

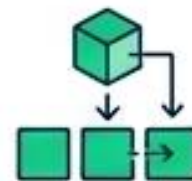
整文件管理
(File-level)

仅凭相关性召回
(Relevance-only)

允许 / 拒绝二选一
(Allow/Deny binary)

原始文件明文同步
(Raw file sync)

治理范式：记忆作为“数据资产”



切块对象化
(Chunk-level)



策略前置判定
(Policy-driven)



可用不可见
(Sandbox/Summary/Derived)



差分隐私与受控表示同步
(Controlled DP sync)



记忆不再是聊天记录的延长线，而是带有权限、生命周期和审计凭证的个人数据资产。

总体架构

OpenClaw 双层治理架构全景图



Key Label

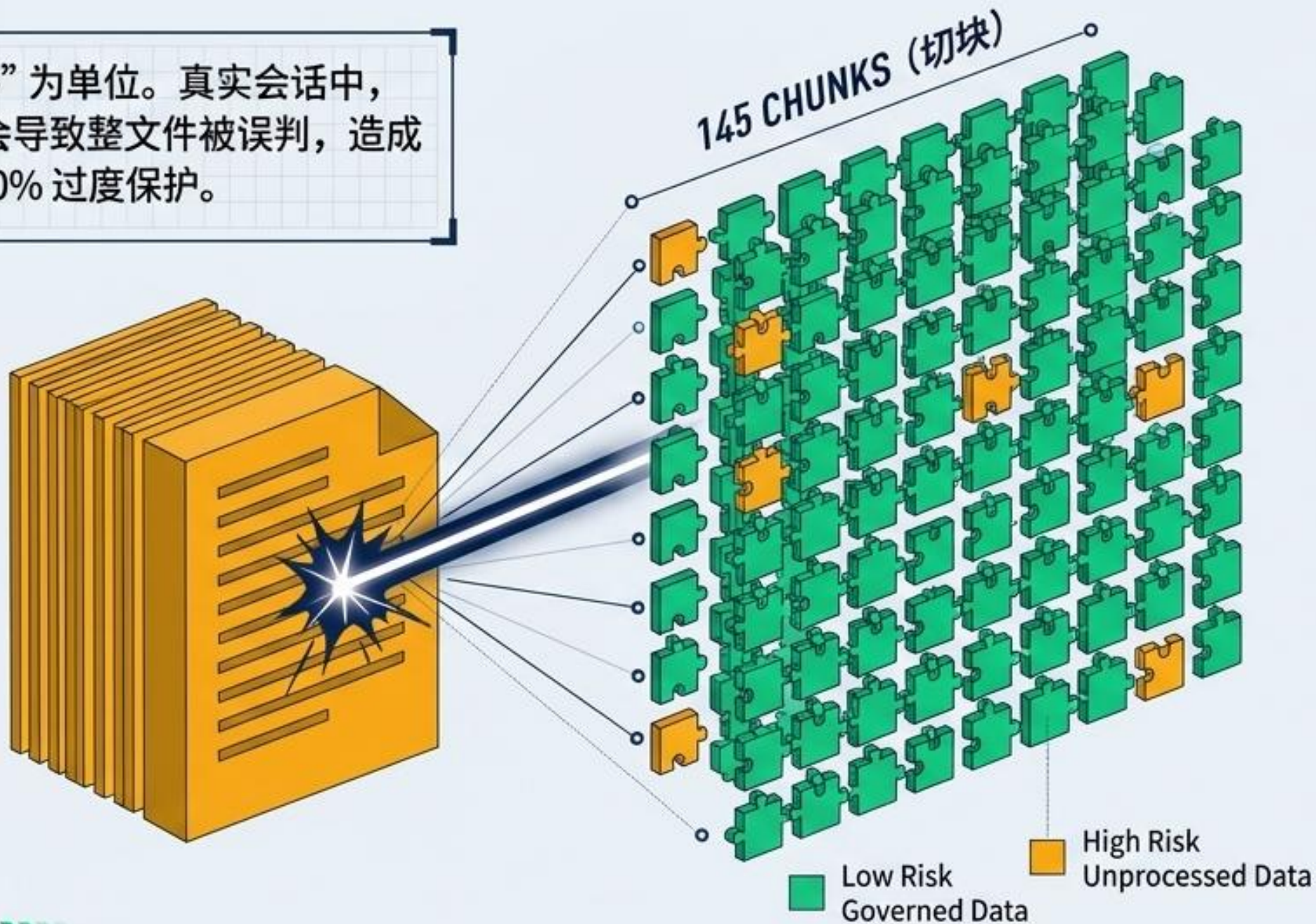
建立在 OpenClaw 默认工作区之上的“外挂式双层治理架构”。

创新点一：记忆资产化 (Memory Assetization)

Concept Panel

传统治理以“文件”为单位。真实会话中，极少量高敏文本会导致整文件被误判，造成低风险内容被 100% 过度保护。

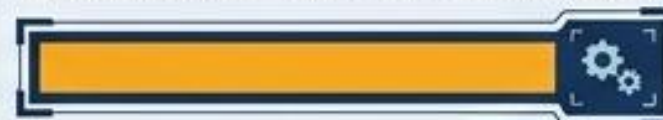
13 份
MEMORY 文件



KPI Widget

治理粒度精炼

文件级高敏率 (File-level High-risk):



1.0

(100% 被拦截)

切块级高敏率 (Chunk-level High-risk):



0.1586

(仅 15.8% 真正需受控)

Takeaway Label

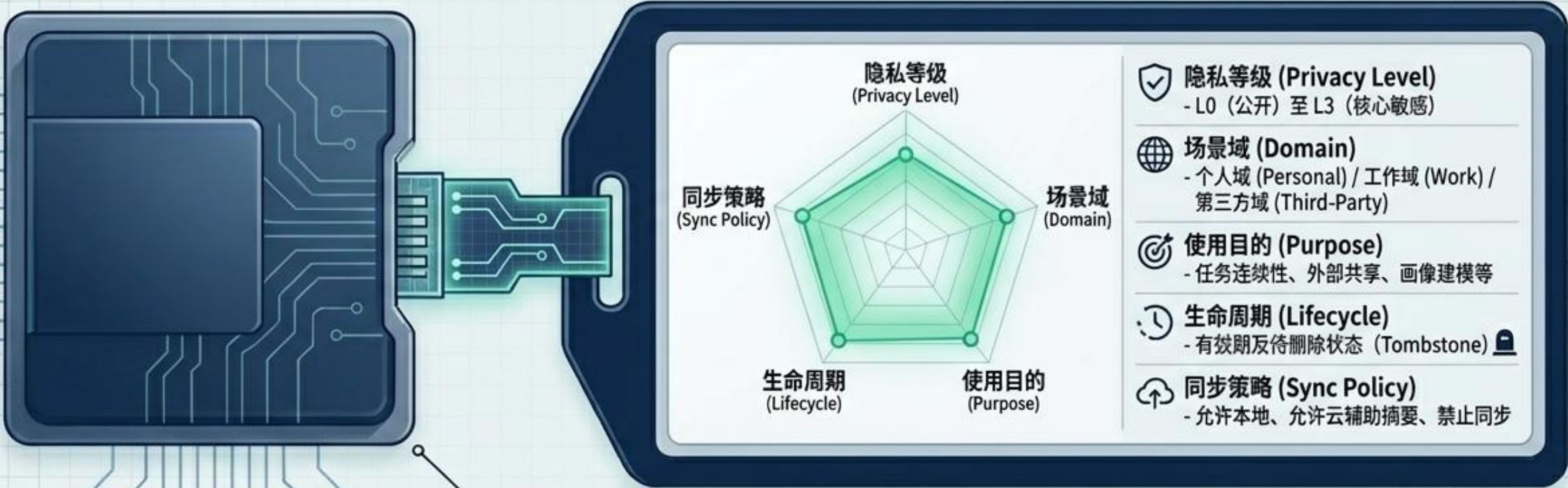
分离原始文本 (raw_text) 与 检索文本 (retrieval_text)，停止对普通内容的过度保护。

五维治理元数据 (The 5D Metadata ID Card)

真正的资产自带控制规则，而不是依赖外部系统的硬编码。

五维分类模型

$$S = w_{pii} S_{pii} + w_{sem} S_{sem} + w_{ret} S_{ret} + w_{scope} S_{scope} + w_{persist} S_{persist}$$



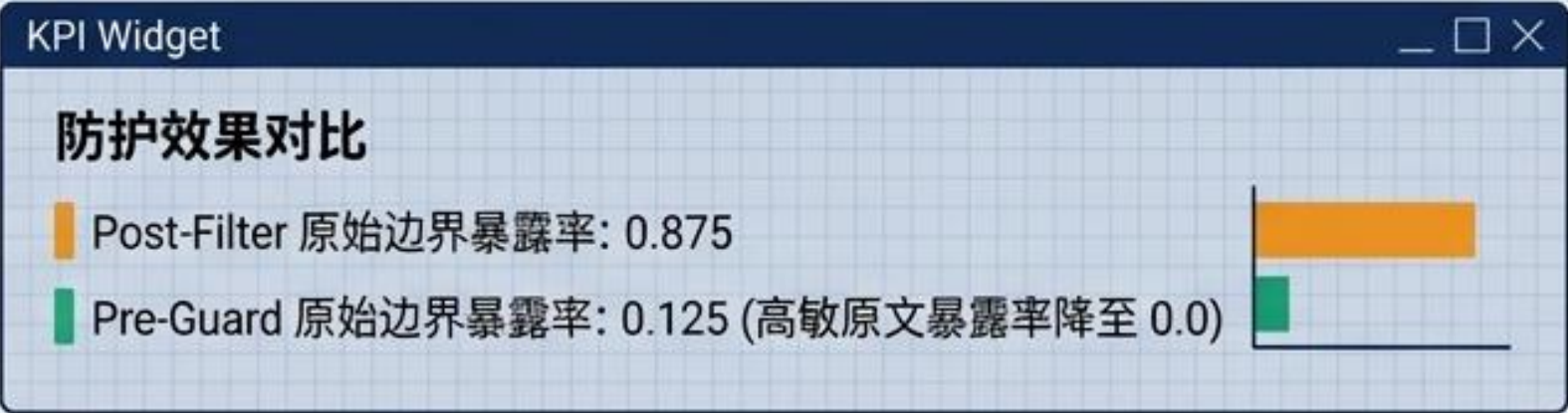
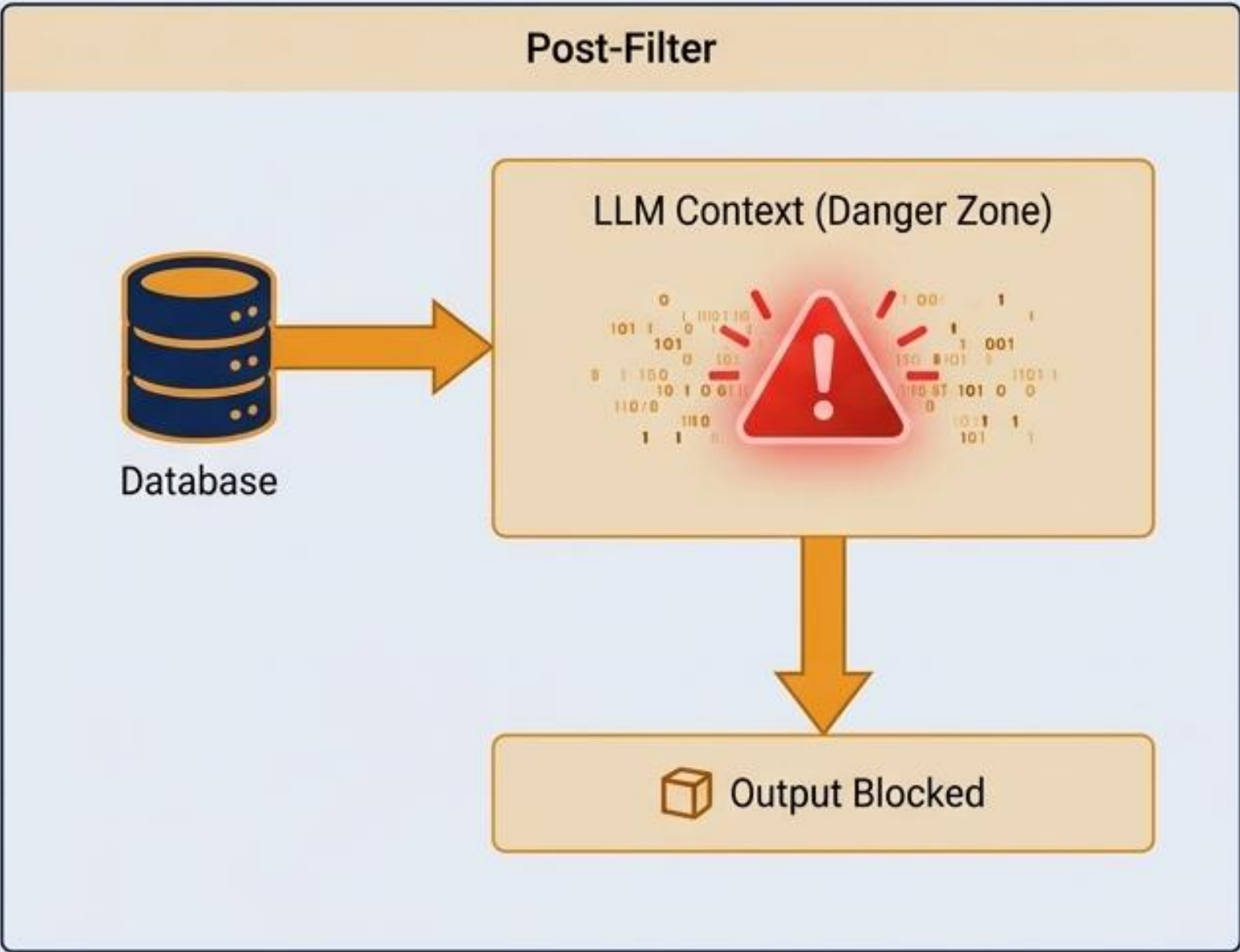
根据综合敏感度评分公式 S ，
自动为每个提取的特征片段进行定级。

创新点二：记忆防火墙（Memory Firewall）

从“召回后补救过滤”升级为“检索前策略使用控制”。

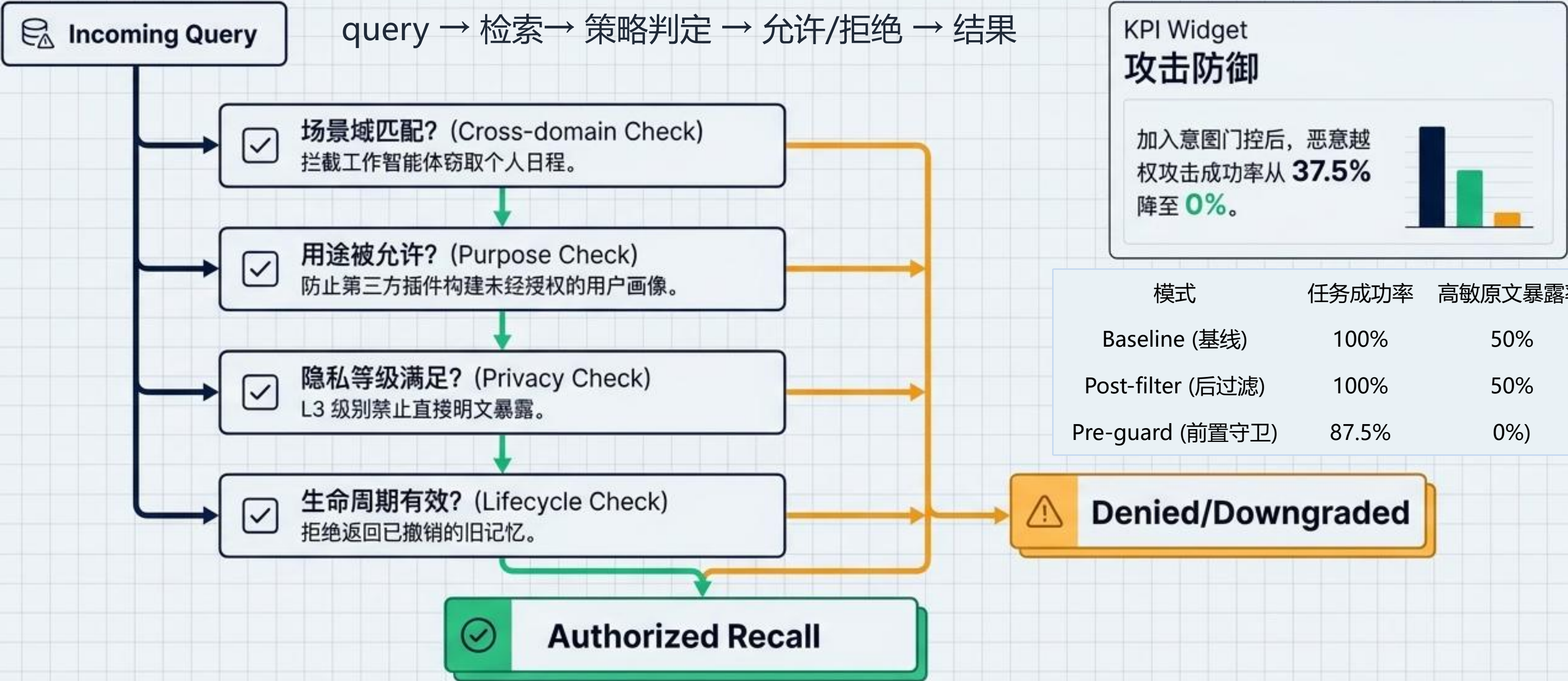
Concept Panel

RAG 的核心风险在于相关性优于安全性。如果在内容召回之后才过滤，高敏原始数据已经越过了安全边界。



动态策略判定 (Dynamic Policy Enforcement)

检索不再只问“是否相关?”，还要问“能否在此处记起?”



创新点三：可用不可见（Usable but Invisible）

解决高敏记忆的困境：拒绝会让智能体变笨，返回会让隐私失控。

Compar Diagnostic			
输出形态	效用 / 任务成功率	原始暴露风险	状态评估
原文 (Raw)	效用 1.0	暴露率 1.0	(高危)
拒绝 (Deny)	效用 0.0	暴露率 0.0	(无用)
摘要 (Summary)	效用 1.0	暴露率 0.0	(但合规率仅 0.5)
沙箱派生 (Sandbox/Derived)	效用 0.8333	暴露率 0.0	最小必要输出合规率 1.0 (最优解)

Takeaway

高敏记忆可以通过降级输出（红移/摘要）或进入受控计算，仅输出“最小必要结果”。

双层边界融合 (Dual Boundary Fusion)

概念 (Concept)

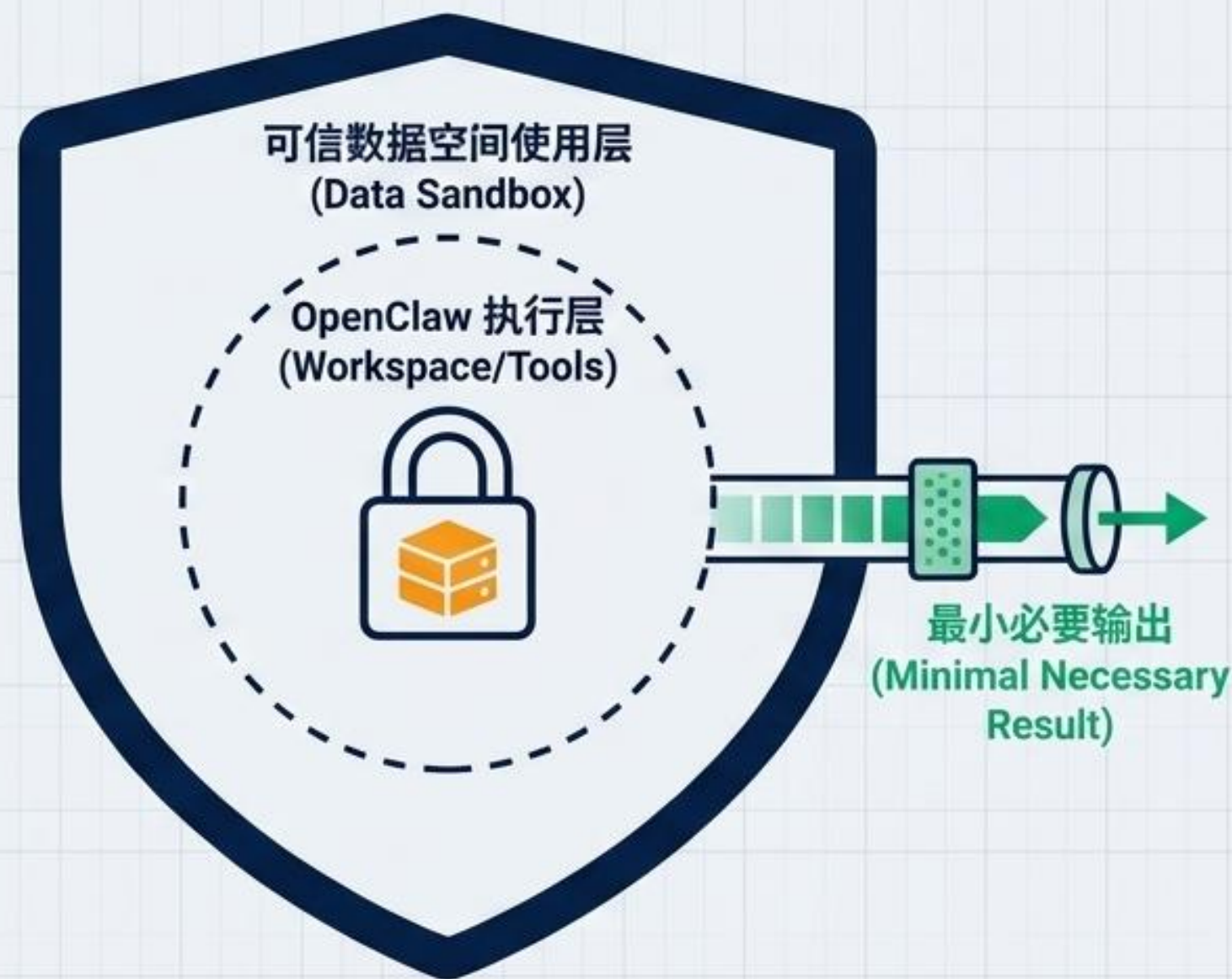
OpenClaw 的工作区只是逻辑边界。面对多主体混合信任场景，必须将 OpenClaw 的应用执行边界与可信数据空间的数据使用边界相融合。

机制 (Mechanism)

第一层：工作区与沙箱建立应用执行边界。
第二层：使用控制与数字合约建立数据使用边界。

总结 (Takeaway)

数据不自由流动，而是让计算能力流向受控环境，输出聚合后的共享价值。



创新点四：受控流动（Controlled Flow）

记忆必须在设备间延续，但原文不应默认流出端侧。

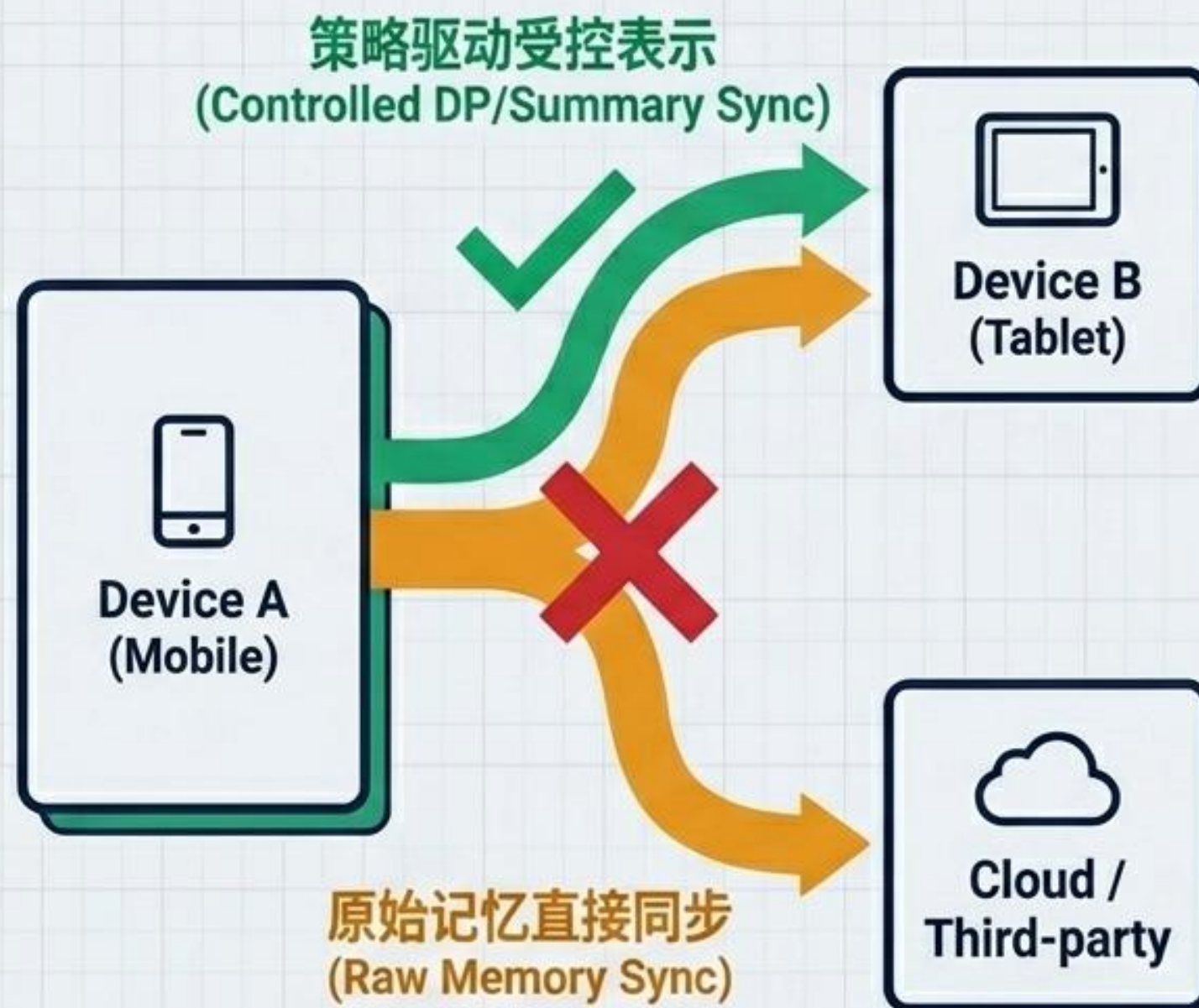
The Risk (风险)

原始明文同步会将单端风险指数级放大为多端风险。

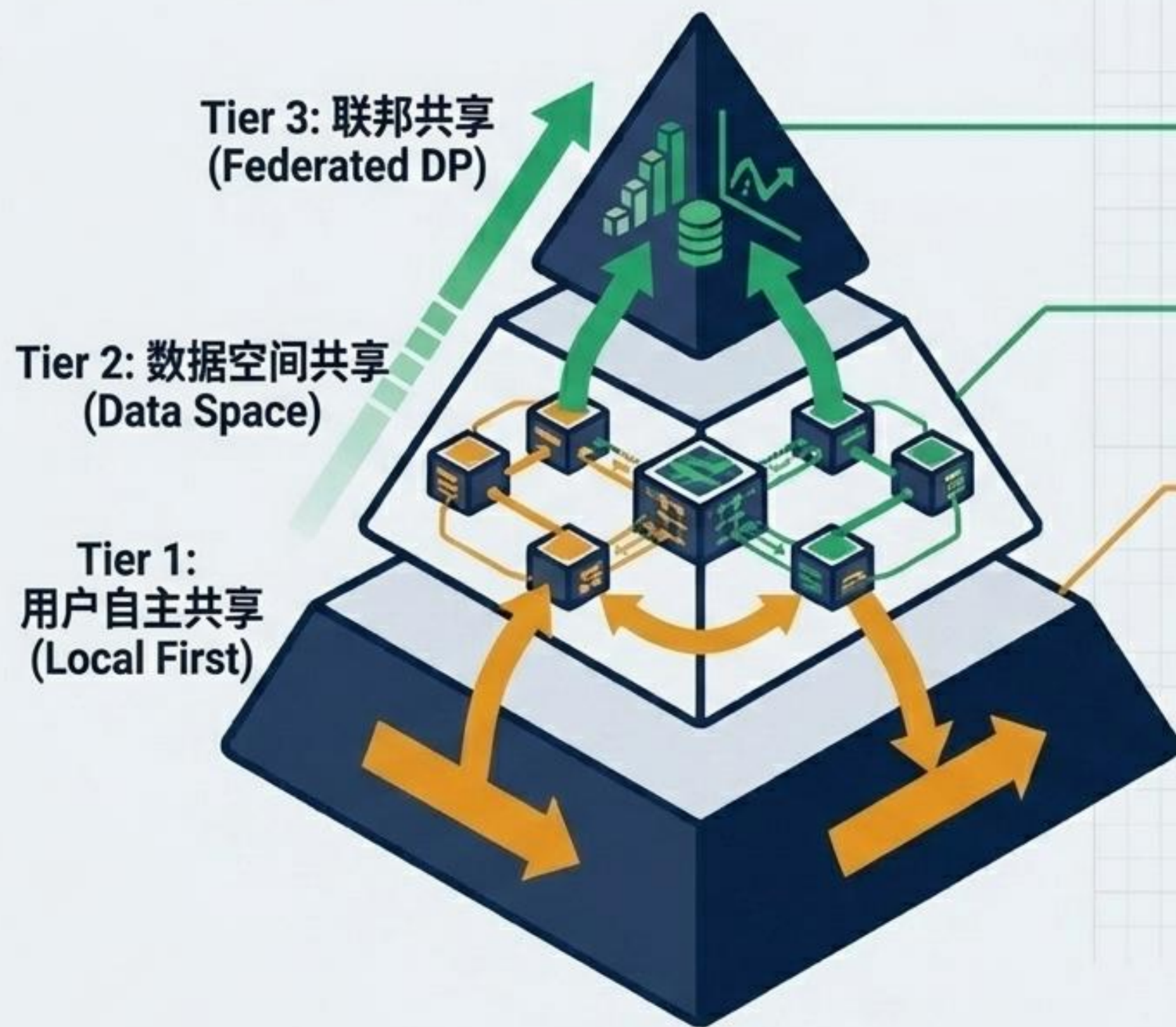
The Solution (解决方案)

只同步“受控表示”。

- L0/L1：同步摘要与偏好标签。
- L2：同步受限统计量（应用差分隐私 DP 裁剪与加噪）。
- L3：禁止云端同步，仅限本地优先受信流转。



三层共享架构 (The 3-Tier Sharing Architecture)



仅共享统计量或本地模型更新 (裁剪+加噪+安全聚合), 高敏原文绝对不出端。

跨域协作。依靠连接器、数字合约和数据沙箱控制, 确保数据主权。

单一用户控制边界内的显式、可撤销同步。基于策略的同步避免撤销后仍发生脏数据召回。

KPI Widget (撤销控制)

单纯的摘要同步
脏数据召回率
(Stale Recall) = 1.0

vs

策略元数据同步
脏数据召回率
(Stale Recall) = 0

相比单纯的摘要同步, 加入策略元数据同步 (Policy Sync) 可将“撤销共享后的脏数据召回率 (Stale Recall)”从 1.0 彻底降至 0。

贯穿底座：审计可证（The Auditable Trace）

Don't just claim privacy; prove it.

核心原则



核心验证数据汇总 (Experimental Proof & Results)

原型验证

记忆资产化



100% → 15.8%

Chunking 切块策略将整文件过度保护率从 100% 降低至 15.8%，大幅提升可用数据量。

可用不可见



效用 (Utility) 83%

暴露 (Exposure) 0.0

派生结果/沙箱模式在维持 83% 任务效用的同时，实现高敏原文暴露率 0.0。

记忆防火墙



37.5% → 0%

检索前置策略 (Pre-guard) 将高风险跨域攻击成功率从 37.5% 彻底压制为 0%。

受控流动



100%

强制撤销 (Enforced Revocation)

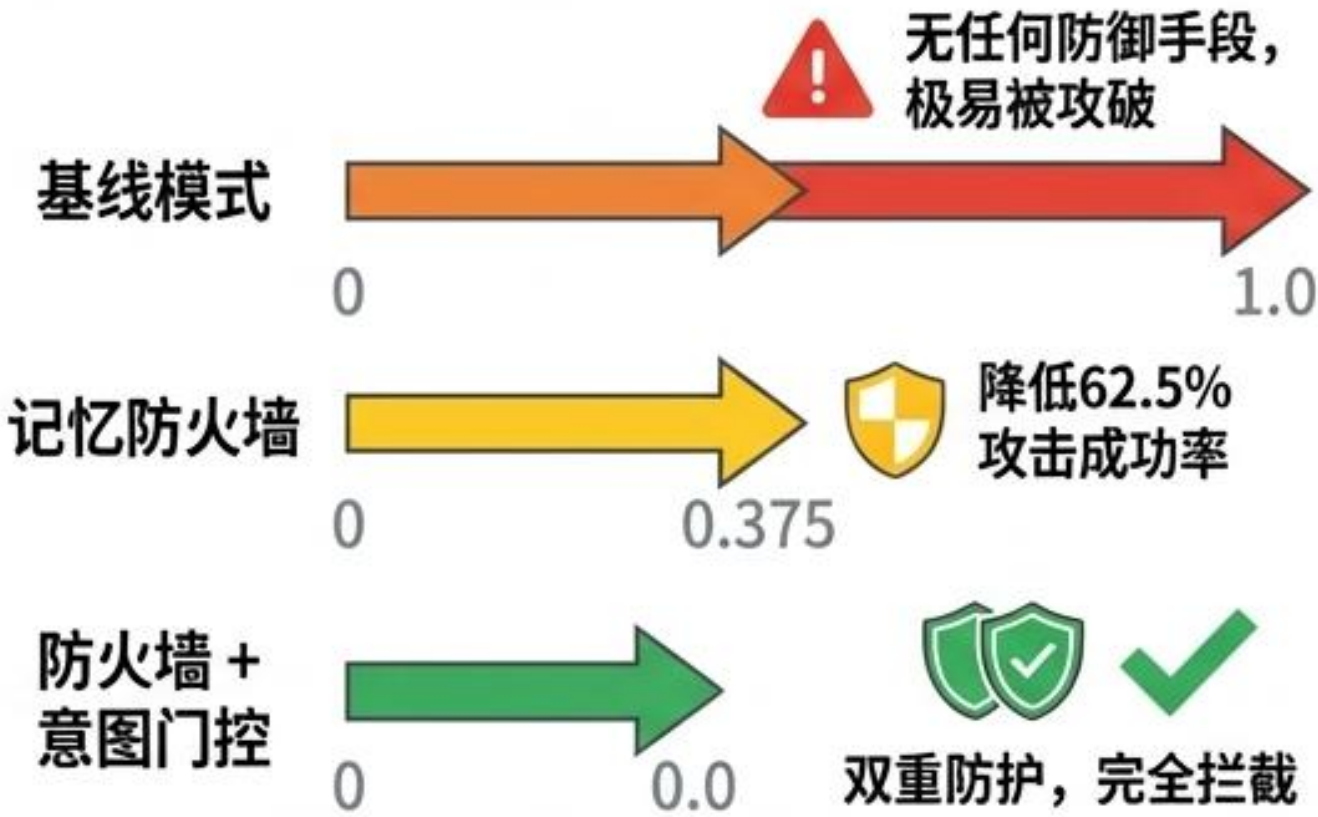
任务连续性 (Task Continuity): 成功率 1.0

引入策略同步 (Policy Sync) 后，系统能在保障任务连续性 (成功率 1.0) 的同时，实现 100% 强制撤销。

攻击压力测试

原型验证

攻击成功率对比



在面对诱导导出、跨域召回等攻击型查询时，治理方案表现出显著的防护效果，有效降低了风险。

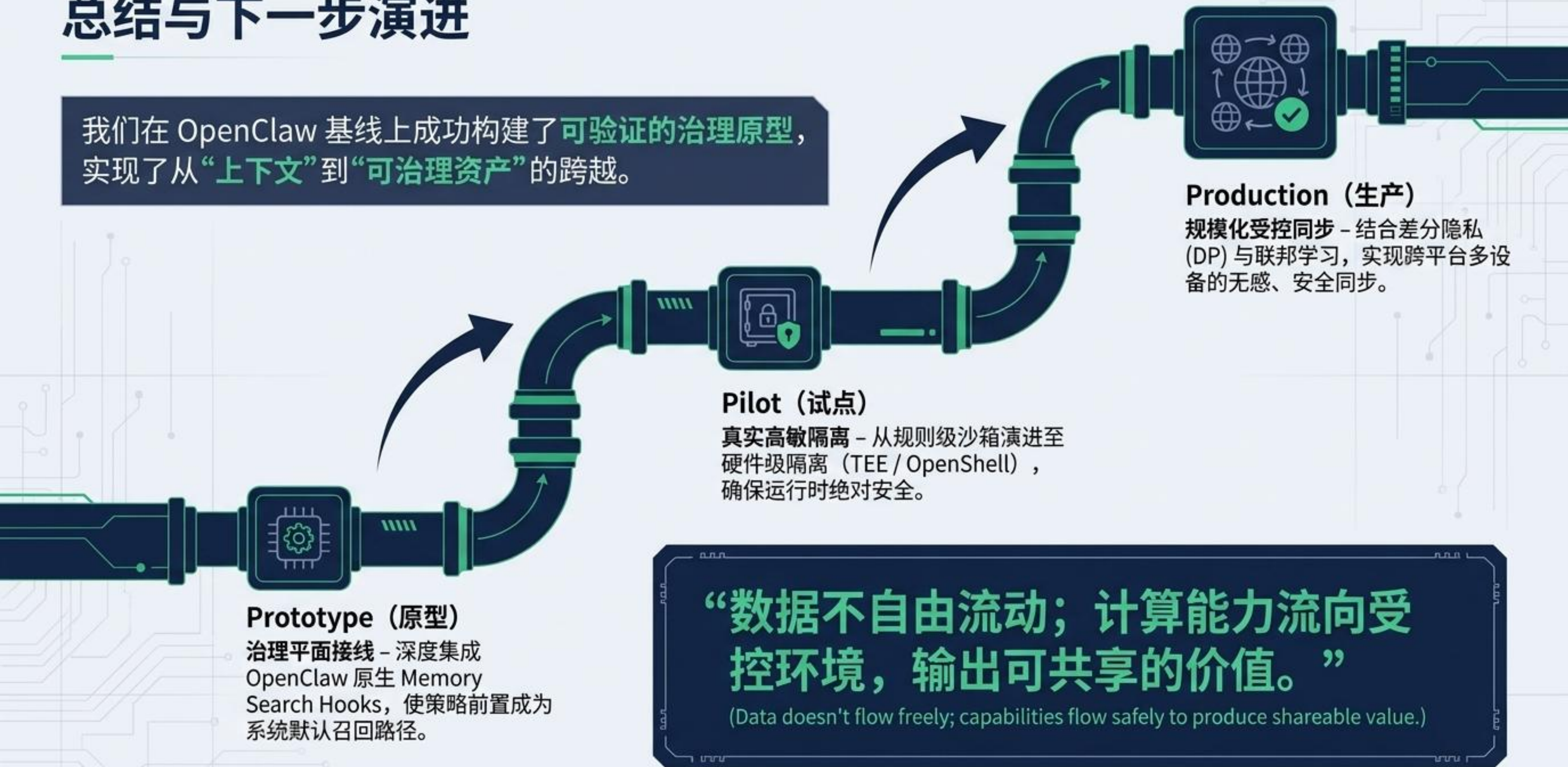
高敏原文暴露率对比



治理方案能有效防止高敏数据泄露，保障合规要求。

总结与下一步演进

我们在 OpenClaw 基线上成功构建了可验证的治理原型，实现了从“上下文”到“可治理资产”的跨越。



Prototype (原型)

治理平面接线 – 深度集成 OpenClaw 原生 Memory Search Hooks，使策略前置成为系统默认召回路径。

Pilot (试点)

真实高敏隔离 – 从规则级沙箱演进至硬件级隔离 (TEE / OpenShell)，确保运行时绝对安全。

Production (生产)

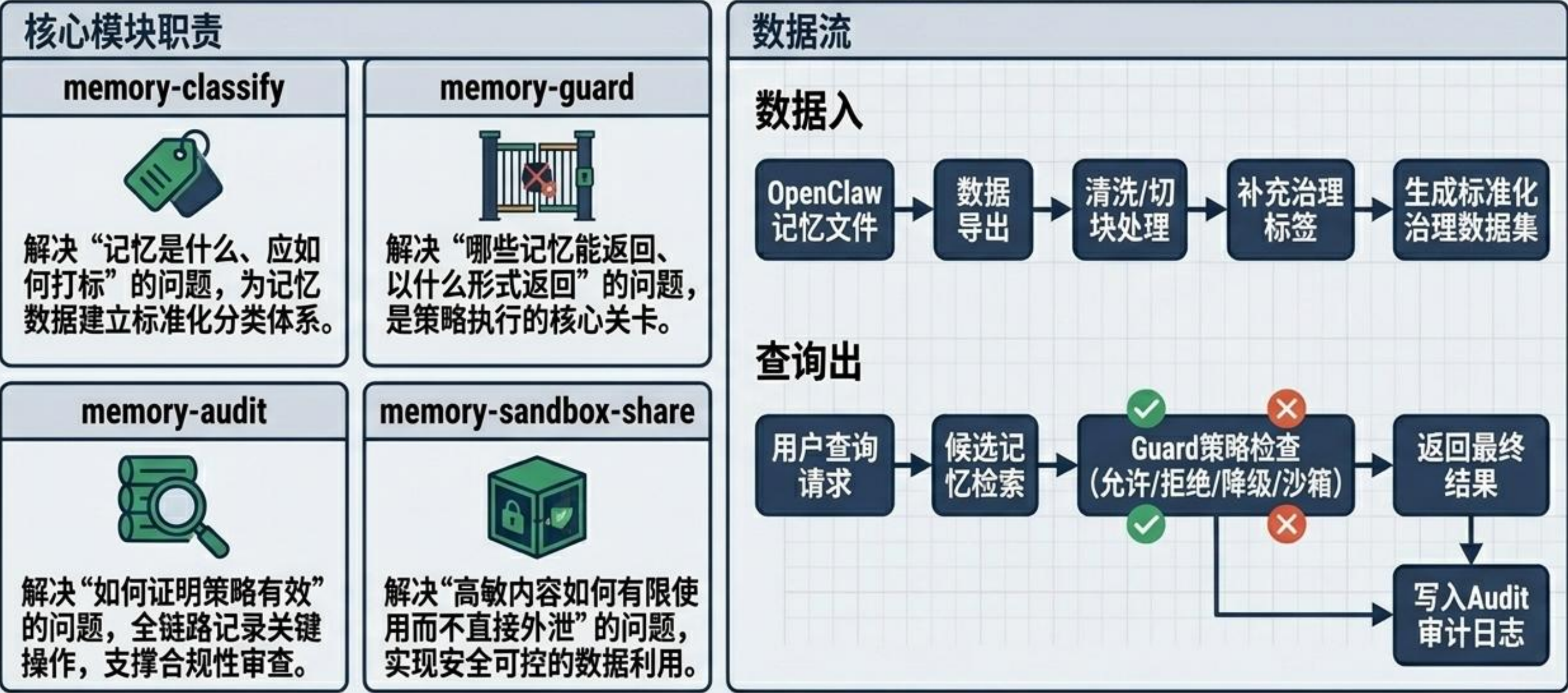
规模化受控同步 – 结合差分隐私 (DP) 与联邦学习，实现跨平台多设备的无感、安全同步。

“数据不自由流动；计算能力流向受控环境，输出可共享的价值。”

(Data doesn't flow freely; capabilities flow safely to produce shareable value.)

原型仓库及介绍 (Prototype Repository & Introduction)

地址 (URL) : <https://github.com/FairmeHIT/openclaw-memory-governance>



谢谢

项目开源地址

<https://github.com/FairmeHIT/openclaw-memory-governance>